

AT 眼镜盒通信协议

AT 眼镜盒通信协议	
[备注]	
密级	内部公开

修订历史			
版本	修订者	日期	备注
1.0	周泽煌	2026-01-21	初稿
1.1	戴杰	2026-03-21	增加机盒心跳/关机/ 盒子ota相关指令
1.2	戴杰	2026-03-26	调整ota相关指令逻辑

1. 协议设计

1.1 帧格式

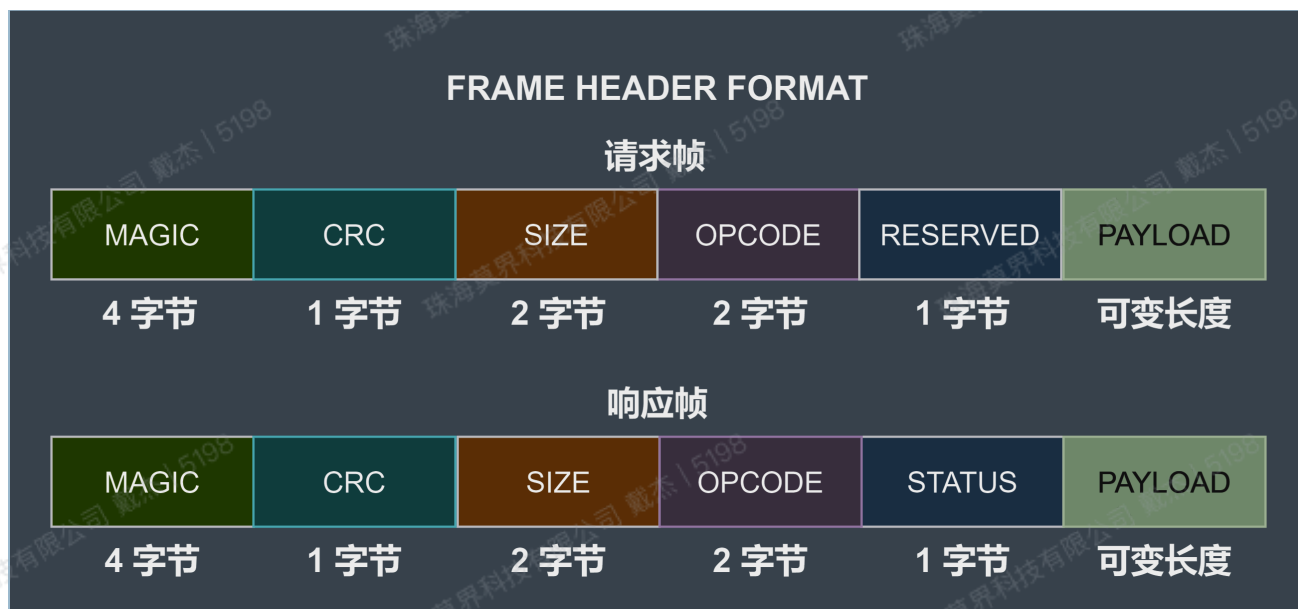


图 3-1 数据帧格式

2.1 字段说明

字段	含义	长度	值	说明
Magic	魔数	4 Bytes	0x23415423	请求码 (#AT#)
			0x23415023	响应码 (#AP#)
Crc	校验码	1 Bytes	0x00 ~ 0xFF	CRC-8校验码
Size	负载大小	2 Bytes	0x0000 ~ 0xFFFF	
Opcode	操作码	2 Bytes	0x0000 ~ 0xFFFF	指令 ID
Reserved	保留字段	1 Bytes	0x00	保留字段
Status	状态码	1 Bytes	0x00	成功
			其它	失败
Payload	数据负载	可变长度		前两个字节用来区分数据流向 参考at_case_role定义

2.2 数据格式

2.2.1 Payload定义

▼ PAYLOAD STRUCT

复制代码

```
// License: Apache 2.0. See LICENSE file in root directory.  
// Copyright(c) 2021 MetaBounds Corporation. All Rights Reserved.
```

```
#ifndef _AT_TYPES_H_  
#define _AT_TYPES_H_
```

```
#ifdef __cplusplus  
extern "C" {  
#endif
```

```
typedef enum  
{  
    AT_CASE_ROLE_CASE,  
    AT_CASE_ROLE_GLASS,  
    AT_CASE_ROLE_MAGBAG,  
    AT_CASE_ROLE_COUNT  
} at_case_role_e;
```

```
typedef enum  
{  
    AT_RW_ACTION_R,  
    AT_RW_ACTION_W,  
    AT_RW_ACTION_COUNT  
} at_rw_action_e;
```

```
typedef enum  
{  
    AT_PACKET_TYPE_MID,  
    AT_PACKET_TYPE_END,  
    AT_PACKET_TYPE_COUNT  
} at_packet_type_e;
```

```
typedef struct  
{  
    uint8_t des; // @see at_case_role_e  
    uint8_t src; // @see at_case_role_e  
} at_case_role;
```

```
typedef struct  
{
```

```

    at_case_role role; // @see at_case_role
    uint32_t size;
} at_case_packet_prepare;

typedef struct
{
    at_case_role role; // @see at_case_role
    uint16_t index; // 包序号
    uint16_t size; // 包大小
} at_case_packet_read;

typedef struct
{
    at_case_role role; // @see at_case_role
    uint16_t index; // 包序号
    uint8_t type; // @see at_packet_type_e
    uint8_t data[];
} at_case_packet_transfer;

typedef struct
{
    at_case_role role; // @see at_case_role
    uint8_t case_soc;
    uint8_t case_sta;
} at_case_data;

typedef struct
{
    at_case_role role; // @see at_case_role
    uint8_t glass_soc;
    uint8_t glass_sta;
    uint8_t case_version; // 眼镜中盒子固件版本
} at_glass_data;

#pragma pack(pop)

#ifdef __cplusplus
}
#endif

```

2.2.2 状态码

状态码枚举

[复制代码](#)

```
typedef enum
{
    AT_SUCCESS                = 0x00, // 成功

    AT_ERR_IO                 = 0x01, // IO 错误
    AT_ERR_TIMEOUT            = 0x02, // 超时
    AT_ERR_LENGTH              = 0x03, // 长度错误
    AT_ERR_PARAMS              = 0x04, // 非法参数
    AT_ERR_RESULT              = 0x05, // 测试失败
    AT_ERR_CRC                 = 0x06, // 校验错误
    AT_ERR_CHANNEL             = 0x07, // 通道错误
    AT_ERR_HARDWARE            = 0x08, // 硬件错误
    AT_ERR_UNSUPPORTED         = 0x09, // 不支持
    AT_ERR_RUNTIME_MODE        = 0x0A, // 运行模式错误

    AT_ERR_DATA_NOT_READY     = 0xFC, // 包不完整或不足以解析
    AT_ERR_NULL                = 0xFD, // 空指针
    AT_ERR_MAGIC               = 0xFE, // 魔数错误
    AT_ERR_UNKNOEN             = 0xFF, // 未知错误
} at_status;
```

状态码	值	说明
AT_SUCCESS	0x00	操作成功
AT_ERR_IO	0x01	IO 错误
AT_ERR_TIMEOUT	0x02	响应、或执行超时
AT_ERR_LENGTH	0x03	长度错误
AT_ERR_PARAMS	0x04	非法参数
AT_ERR_RESULT	0x05	指令执行失败
AT_ERR_CRC	0x06	CRC 校验错误
AT_ERR_CHANNEL	0x07	通道传输错误
AT_ERR_HARDWARE	0x08	硬件错误
.....		
.....		


```
#endif
```

✓ at_crc.c

复制代码

```
#include "at_crc.h"
```

```
static const uint8_t crc8_table[256] = {  
    0x00, 0x31, 0x62, 0x53, 0xC4, 0xF5, 0xA6, 0x97, 0xB9, 0x88, 0xDB, 0xEA, 0x7D, 0x4C, 0x1F, 0x2E,  
    0x43, 0x72, 0x21, 0x10, 0x87, 0xB6, 0xE5, 0xD4, 0xFA, 0xCB, 0x98, 0xA9, 0x3E, 0x0F, 0x5C, 0x6D,  
    0x86, 0xB7, 0xE4, 0xD5, 0x42, 0x73, 0x20, 0x11, 0x3F, 0x0E, 0x5D, 0x6C, 0xFB, 0xCA, 0x99, 0xA8,  
    0xC5, 0xF4, 0xA7, 0x96, 0x01, 0x30, 0x63, 0x52, 0x7C, 0x4D, 0x1E, 0x2F, 0xB8, 0x89, 0xDA, 0xEB,  
    0x3D, 0x0C, 0x5F, 0x6E, 0xF9, 0xC8, 0x9B, 0xAA, 0x84, 0xB5, 0xE6, 0xD7, 0x40, 0x71, 0x22, 0x13,  
    0x7E, 0x4F, 0x1C, 0x2D, 0xBA, 0x8B, 0xD8, 0xE9, 0xC7, 0xF6, 0xA5, 0x94, 0x03, 0x32, 0x61, 0x50,  
    0xBB, 0x8A, 0xD9, 0xE8, 0x7F, 0x4E, 0x1D, 0x2C, 0x02, 0x33, 0x60, 0x51, 0xC6, 0xF7, 0xA4, 0x95,  
    0xF8, 0xC9, 0x9A, 0xAB, 0x3C, 0x0D, 0x5E, 0x6F, 0x41, 0x70, 0x23, 0x12, 0x85, 0xB4, 0xE7, 0xD6,  
    0x7A, 0x4B, 0x18, 0x29, 0xBE, 0x8F, 0xDC, 0xED, 0xC3, 0xF2, 0xA1, 0x90, 0x07, 0x36, 0x65, 0x54,  
    0x39, 0x08, 0x5B, 0x6A, 0xFD, 0xCC, 0x9F, 0xAE, 0x80, 0xB1, 0xE2, 0xD3, 0x44, 0x75, 0x26, 0x17,  
    0xFC, 0xCD, 0x9E, 0xAF, 0x38, 0x09, 0x5A, 0x6B, 0x45, 0x74, 0x27, 0x16, 0x81, 0xB0, 0xE3, 0xD2,  
    0xBF, 0x8E, 0xDD, 0xEC, 0x7B, 0x4A, 0x19, 0x28, 0x06, 0x37, 0x64, 0x55, 0xC2, 0xF3, 0xA0, 0x91,  
    0x47, 0x76, 0x25, 0x14, 0x83, 0xB2, 0xE1, 0xD0, 0xFE, 0xCF, 0x9C, 0xAD, 0x3A, 0x0B, 0x58, 0x69,  
    0x04, 0x35, 0x66, 0x57, 0xC0, 0xF1, 0xA2, 0x93, 0xBD, 0x8C, 0xDF, 0xEE, 0x79, 0x48, 0x1B, 0x2A,  
    0xC1, 0xF0, 0xA3, 0x92, 0x05, 0x34, 0x67, 0x56, 0x78, 0x49, 0x1A, 0x2B, 0xBC, 0x8D, 0xDE, 0xEF,  
    0x82, 0xB3, 0xE0, 0xD1, 0x46, 0x77, 0x24, 0x15, 0x3B, 0x0A, 0x59, 0x68, 0xFF, 0xCE, 0x9D, 0xAC,  
};
```

```
#ifdef META_CRC_16
```

```
static const uint8_t crc8_reverse_table[16] = {  
    0x00, 0x08, 0x04, 0x0C, 0x02, 0x0A, 0x06, 0x0E,
```

```

    0x01, 0x09, 0x05, 0x0D, 0x03, 0x0B, 0x07, 0x0F,
};
#endif

uint8_t at_crc8(uint8_t *ptr, uint16_t len, uint8_t crc_origin)
{
    while (len--) {
        crc_origin = crc8_table[*ptr++ ^ crc_origin];
    }

    return crc_origin;
}

```

4. 指令格式

4.1 机盒心跳

	Opcode	Payload
REQ	AT_OPCODE_CASE_HEA	at_case_data
RSP	RT	at_glass_data

4.2 关机

	Opcode	Payload
REQ	AT_OPCODE_CASE_SH	at_case_role
RSP	UTDOWN	at_case_role

4.3 传输固件准备

	Opcode	Payload
--	--------	---------

REQ	AT_OPCODE_CASE_PAC KET_PREPARE	at_case_role
RSP		at_case_packet_prepare

4.4 传输固件

	Opcode	Payload
REQ	AT_OPCODE_CASE_PAC KET_READ	at_case_packet_read
RSP		at_case_packet_transfer